

1. 분석 및 기술 선택

1.1 게임 엔진 선택: Unity

옵션 분석

엔진	장점	단점
Unity	넓은 VR 생태계, C#, 크로스 플랫폼	대규모 프로젝트 유료 라이선스
Unreal Engine	우수한 그래픽, C++	학습 어려움, 무거움

선택 근거

Unity 6000.2.14f1을 선택한 이유:

- 1. 성숙한 VR 생태계 - 공식 XR Interaction Toolkit, 네이티브 OpenXR 지원
- 2. 크로스 플랫폼 - Quest, PCVR, Desktop 단일 빌드
- 3. 통합 WebRTC - 공식 com.unity.webrtc 패키지
- 4. 활발한 커뮤니티 - 풍부한 문서, 일반적인 문제 해결책
- 5. C# 언어 - 높은 생산성, 강한 타이핑, async/await

1.2 네트워크 프로토콜 선택: WebSocket

옵션 분석

프로토콜	지연 시간	신뢰성	복잡도
WebSocket	낮음	TCP 보장	중간
순수 UDP	매우 낮음	보장 없음	높음

선택 근거

WebSocket을 선택한 이유:

- 1. 양방향 - 클라이언트-서버 양방향 통신

- 2. **영구적** - 연결 유지, 메시지마다 재연결 불필요
- 3. **호환성** - 모든 플랫폼에서 설정 없이 작동
- 4. **단순성** - 패킷 관리가 필요한 UDP보다 빠른 구현

1.3 음성 통신 선택: WebRTC

근거

음성은 다음이 필요합니다:

- **낮은 지연 시간** (자연스러운 대화를 위해 $< 150\text{ms}$)
- **P2P** (서버 부하 감소)

WebRTC는 이러한 기준에 완벽히 부합:

- 실시간 오디오/비디오용 프로토콜
- 클라이언트 간 직접 P2P 연결
- 최적화된 오디오 코덱 (Opus)
- STUN/TURN을 통한 NAT 통과

1.4 클라이언트-서버 vs P2P 아키텍처

선택: 하이브리드

아키텍처 :

WebSocket (서버 경유)

- 위치/회전
- 룸 관리
- 채팅/메시지
- 화이트보드
- 파일 공유

WebRTC (P2P 직접)

- 음성 오디오
- (시그널링만 서버 경유)

근거

데이터	서버 경유 (WebSocket)	P2P (WebRTC)
위치	✓ (모든 사용자에게 브로드캐스트 필요)	
음성		✓ (지연 시간 중요)
화이트보드	✓ (영구성, 후발 참가자)	
파일	✓ (신뢰성 필요)	

1.5 기술 요약표

컴포넌트	기술	근거
엔진	Unity 6000.2.14f1	VR 생태계, C#, 크로스 플랫폼
렌더링	URP 17.2.0	VR 성능, 현대적 효과
VR 프레임워크	XR Interaction Toolkit 3.2.2	Unity 표준, 잘 문서화됨
VR 런타임	OpenXR 1.16.1	개방형 표준, 멀티 헤드셋
네트워크 동기화	NativeWebSocket	양방향, 신뢰성, 단순
음성	Unity WebRTC 3.0.0	P2P, 낮은 지연, 공간 오디오
백엔드	Node.js + ws	가벼움, 네이티브 비동기, npm 생태계
데이터베이스	MariaDB (선택)	오픈 소스, MySQL 호환
인증	bcrypt + JWT	표준 보안, 상태 비저장

2. 프로젝트 아키텍처

2.1 Unity 씬 아키텍처

근거

프로젝트는 책임 분리를 위해 **2개 씬** 아키텍처를 사용합니다:

씬 0: Bootstrap (영구적)

시작 시 로드, 항상 메모리에 유지.
싱글톤과 전역 매니저 포함.

NetworkManager (DontDestroyOnLoad)
RoomManager
GameManager
VoiceChatManager
AuthManager
SoundManager
MainMenuUI

씬 1: Meet (추가 로드)

연결 후 추가 로드.
3D 환경과 게임 오브젝트 포함.

Lobby
MeetingRoomA
MeetingRoomB
Whiteboards
SpectatorCamera

이 아키텍처의 장점

- 1. **영구성** - 네트워크 매니저가 씬 변경에도 유지

- 2. 분리 - 로직 (Bootstrap) vs 콘텐츠 (Meet)
- 3. 빠른 로딩 - Meet 실행 추가 로드, 완전 리로드 없음
- 4. 유지보수 - 네트워크 수정 없이 환경 변경 가능

3. 개발 히스토리

3.1 개발 일정

1단계: 기반 구축

- Unity 프로젝트 초기화
- XR Interaction Toolkit 패키지 설정
- 기본 네트워크 시스템 생성 (VRNetworkManager)
- WebSocket 룸 시스템 구현
- 첫 번째 플레이어 프리팹 (로컬 및 원격)

2단계: 인터페이스 및 내비게이션

- 로비 및 회의실 생성 (Room A, B)
- 텔레포트 구현
- VR 사용자 인터페이스 (패널, 버튼)
- 스폰 시스템 및 위치 관리

3단계: 통신

- 음성 채팅을 위한 WebRTC 통합
- 기본 화이트보드 구현
- 첫 번째 동기화 테스트

4단계: 화이트보드 및 동기화

- 화이트보드 동기화 수정
- 다중 사용자 그리기 구현
- 네트워크 성능 최적화

5단계: 데스크톱 모드 및 공유

- 데스크톱 모드 추가 (키보드/마우스)
- WebRTC를 통한 화면 공유
- 파일 공유 (PDF, 이미지, 문서)
- 아바타 커스터마이징

6단계: 최적화 및 마무리

- 성능 최적화 (배칭, 캐시)
- 메모리 누수 수정
- WebRTC용 TURN/STUN 설정
- 레이저 포인터 추가

7단계: 메인 메뉴 및 옵션

- 메인 메뉴 생성
- 설정 시스템 (오디오, 그래픽, VR)
- 로딩 화면
- 새 아바타 모델

8단계: 녹화 및 인증

- 회의 녹화 시스템
- 인증 구현 (로그인/회원가입)
- 멀미 방지를 위한 VR 최적화
- 앰비언스 사운드 및 뮤트 존
- 사용자 인터페이스 최종 마무리

3.2 개발 도구

도구	용도
Unity Editor	메인 개발
Visual Studio / Rider	C# 편집
Git	버전 관리
Node.js	백엔드 서버

4. 기술적 구현

4.1 룸 시스템

구현

- **룸 코드:** 6자리 고유 영숫자
- **호스트 권한:** 생성자가 호스트
- **브로드캐스트 범위:** 메시지가 룸으로 제한

네트워크 메시지

```
// 룸 생성
{ type: "room-available", data: { roomId, roomName, roomType, maxPlayers } }

// 룸 입장
{ type: "room-join", data: { roomId, playerName } }

// 룸 퇴장
{ type: "room-leave", data: { roomId } }
```

4.2 플레이어 동기화

주파수 및 최적화

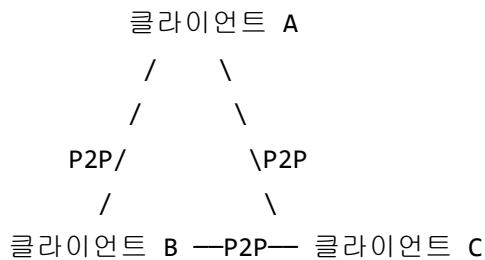
- **30 Hz** 업데이트 (초당 30개 메시지)
- **이동 임계값:** 0.01m / 1° (정지 시 전송 안 함)
- **보간:** 수신된 움직임 스무딩

동기화되는 데이터

```
public class VRPositionData
{
    public float[] headPos;      // 머리 위치 [x, y, z]
    public float[] headRot;      // 머리 회전 [x, y, z, w]
    public float[] leftHandPos;  // 왼손 위치
    public float[] leftHandRot;  // 왼손 회전
    public float[] rightHandPos; // 오른손 위치
    public float[] rightHandRot; // 오른손 회전
}
```

4.3 WebRTC 음성 통신

메시 토폴로지



각 클라이언트는 룸의 다른 모든 클라이언트와 직접 P2P 연결을 설정합니다.

연결 과정

1. **시그널링** (WebSocket 서버 경유)
 - 클라이언트 A가 클라이언트 B에게 offer 전송
 - 클라이언트 B가 answer 로 응답
 - ICE candidates (네트워크 경로) 교환
2. **P2P 연결**
 - 시그널링 완료 후 직접 연결
 - 오디오가 서버를 거치지 않고 P2P로 전송

3D 공간 오디오

각 참가자의 오디오는 3D 공간에서 머리 위치에 부착되어 자연스러운 공간화를 만듭니다.

4.4 협업 화이트보드

3계층 아키텍처

- 레이어 3: 그리기 도구

(WhiteboardMarker / DesktopDrawer)

→ 입력 캡처, 로컬 그리기
- 레이어 2: 네트워크 그리기 표면

(WhiteboardDrawingSurface)

→ 네트워크에서 선 수신

→ 투명, 오버레이
- 레이어 1: 화이트보드 배경

(Whiteboard)

→ 흰색 배경

→ 프레젠테이션 모드 (이미지)

동기화

- 배치: 선이 일괄로 전송됨 (33ms)
- 후발 참가자: 현재 상태 수신을 위한 Request/State 패턴

5. API 및 이벤트 문서

5.1 핵심 이벤트 시스템

VRNetworkManager 이벤트

이벤트	시그니처	발생 시점
OnConnected	Action	서버 연결 성공
OnDisconnected	Action	서버 연결 해제
OnPeerConnected	Action<string>	새 피어 연결 (peerId)

이벤트	시그니처	발생 시점
OnPeerDisconnected	Action<string>	피어 연결 해제 (peerId)
OnMessageReceived	Action<NetworkMessage>	메시지 수신
OnConnectionError	Action<string>	연결 오류 (errorMsg)

VRRoomManager 이벤트

이벤트	시그니처	발생 시점
OnRoomCreated	Action<string>	룸 생성 완료 (roomId)
OnRoomJoined	Action<string>	룸 입장 완료 (roomId)
OnRoomLeft	Action	룸 퇴장 완료
OnPlayerJoined	Action<VRPlayerData>	새 플레이어 입장
OnPlayerLeft	Action<string>	플레이어 퇴장 (playerId)
OnRoomTypeChanged	Action<RoomType>	룸 타입 변경
OnAvatarUpdated	Action<string, AvatarData>	아바타 업데이트

VRGameManager 이벤트

이벤트	시그니처	발생 시점
OnLocalPlayerSpawned	Action<GameObject>	로컬 플레이어 스폰
OnRemotePlayerSpawned	Action<string, GameObject>	원격 플레이어 스폰
OnRemotePlayerDespawned	Action<string>	원격 플레이어 제거

VoiceChatManager 이벤트

이벤트	시그니처	발생 시점
OnVoiceChatReady	Action	음성 채팅 초기화 완료
OnPeerVoiceConnected	Action<string>	피어 음성 연결

이벤트	시그니처	발생 시점
OnPeerVoiceDisconnected	Action<string>	피어 음성 해제

RecordingManager 이벤트

이벤트	시그니처	발생 시점
OnRecordingStarted	Action	녹화 시작
OnRecordingStopped	Action<string>	녹화 종료 (filePath)
OnStateChanged	Action<RecordingState>	상태 변경
OnMarkerAdded	Action<RecordingMarker>	마커 추가

이벤트 구독 패턴

```
public class ExampleScript : MonoBehaviour
{
    private void OnEnable()
    {
        // 이벤트 구독
        VRRoomManager.Instance.OnPlayerJoined += HandlePlayerJoined;
        VRRoomManager.Instance.OnPlayerLeft += HandlePlayerLeft;
    }

    private void OnDisable()
    {
        // 이벤트 구독 해제 (메모리 누수 방지)
        if (VRRoomManager.Instance != null)
        {
            VRRoomManager.Instance.OnPlayerJoined -= HandlePlayerJoined;
            VRRoomManager.Instance.OnPlayerLeft -= HandlePlayerLeft;
        }
    }

    private void HandlePlayerJoined(VRPlayerData playerData)
    {
        Debug.Log($"플레이어 입장: {playerData.playerName}");
    }

    private void HandlePlayerLeft(string playerId)
    {
        Debug.Log($"플레이어 퇴장: {playerId}");
    }
}
```

5.2 네트워크 메시지 프로토콜

기본 메시지 구조

```
[Serializable]
public class NetworkMessage
{
    public string type;        // 메시지 타입
    public string senderId;    // 발신자 ID
    public string data;        // JSON 직렬화된 데이터
}
```

중요: Unity의 JsonUtility 는 중첩 객체를 지원하지 않습니다.
모든 데이터는 평면 구조로 직렬화해야 합니다.

전체 메시지 타입 목록

연결 관리

타입	방향	데이터	설명
welcome	S→C	{ peerId }	연결 성공, 클라이언트 ID 할당
peer-connected	S→C	{ peerId }	새 피어 연결 알림
peer-disconnected	S→C	{ peerId }	피어 연결 해제 알림

룸 관리

타입	방향	데이터	설명
room-create	C→S	{ roomType, playerName }	룸 생성 요청
room-available	S→C	{ roomId, roomType, hostId }	룸 생성 완료
room-join	C→S	{ roomId, playerName }	룸 입장 요청
room-welcome	S→C	{ roomId, players[], roomType }	입장 성공 + 기존 플레이어 목록
room-leave	C→S	{ roomId }	룸 퇴장

타입	방향	데이터	설명
room-list	S→C	{ rooms[] }	사용 가능한 룸 목록
room-teleport	C→S/S→C	{ roomType }	룸 내 텔레포트
player-name-update	C→S/S→C	{ playerName }	이름 변경
avatar-update	C→S/S→C	{ colors, modelIndex }	아바타 업데이트

VR 동기화

타입	방향	데이터	설명
vr-position	C→S→C	VRPositionData	위치/회전 동기화 (30Hz)

```
[Serializable]
public class VRPositionData
{
    public string visitorId;
    public float[] headPos;      // [x, y, z]
    public float[] headRot;      // [x, y, z, w] (Quaternion)
    public float[] leftHandPos;
    public float[] leftHandRot;
    public float[] rightHandPos;
    public float[] rightHandRot;
}
```

음성 통신 (WebRTC 시그널링)

타입	방향	데이터	설명
webrtc-offer	C→S→C	{ targetId, sdp }	SDP Offer
webrtc-answer	C→S→C	{ targetId, sdp }	SDP Answer
webrtc-ice-candidate	C→S→C	{ targetId, candidate }	ICE 후보

화이트보드

타입	방향	데이터	설명
whiteboard-batch	C→S→C	{ lines[], boardId }	선 배치 전송

타입	방향	데이터	설명
whiteboard-clear	C→S→C	{ boardId }	화이트보드 초기화
whiteboard-request	C→S	{ boardId }	현재 상태 요청
whiteboard-state	S→C	{ lines[], boardId }	전체 상태 응답

공유 기능

타입	방향	데이터	설명
screen-share-start	C→S→C	{ }	화면 공유 시작
screen-share-frame	C→S→C	{ imageData }	프레임 (JPEG Base64)
screen-share-stop	C→S→C	{ }	화면 공유 종료
file-share-start	C→S→C	{ fileName, fileSize }	파일 공유 시작
file-share-chunk	C→S→C	{ data, index }	파일 청크
file-share-complete	C→S→C	{ }	파일 전송 완료
file-present-start	C→S→C	{ fileName }	프레젠테이션 시작
file-present-page	C→S→C	{ pageNumber }	페이지 변경
file-present-stop	C→S→C	{ }	프레젠테이션 종료
laser-pointer	C→S→C	{ position, direction, active }	레이저 포인터

녹화

타입	방향	데이터	설명
recording-status	C→S→C	{ isRecording, hostId }	녹화 상태
recording-marker	C→S→C	{ type, timestamp, note }	마커 추가

인증

타입	방향	데이터	설명
auth-login	C→S	{ email, password }	로그인 요청

타입	방향	데이터	설명
auth-register	C→S	{ email, password, username }	회원가입 요청
auth-verify	C→S	{ token }	토큰 검증
auth-logout	C→S	{ }	로그아웃
auth-response	S→C	{ success, token, error }	인증 응답

6. 결과 및 시연

6.1 구현된 기능

기능	상태	설명
멀티플레이어 연결	✔ 완료	자동 재연결 WebSocket
룸 시스템	✔ 완료	생성, 입장, 퇴장, 강퇴
VR 동기화	✔ 완료	30Hz, 머리 + 양손
음성 통신	✔ 완료	WebRTC P2P, 공간 오디오
데스크톱 모드	✔ 완료	WASD + 마우스
화이트보드	✔ 완료	VR/데스크톱 그리기, 동기화
화면 공유	✔ 완료	854x480 @ 3fps
파일 공유	✔ 완료	PDF, 이미지, 문서
레이저 포인터	✔ 완료	VR (A) / 데스크톱 (L)
인증	✔ 완료	로그인/회원가입/게스트
아바타 커스터마이징	✔ 완료	색상
녹화	✔ 완료	1080p, 비동기 파이프라인
VR 메뉴	✔ 완료	전체 인터페이스
오디오 시스템	✔ 완료	앰비언스, 효과음

6.2 성능

지표	값
평균 네트워크 지연	< 50ms (로컬)
VR 프레임레이트	72 FPS 안정
서버 CPU 사용률	< 5% (10 클라이언트)
클라이언트당 대역폭	~50 KB/s

7. 배포 및 운영

7.1 배포 가이드

서버 배포 (프로덕션)

요구 사항

- Node.js 18+ LTS
- MariaDB 10.6+ (선택)
- SSL 인증서 (Let's Encrypt 권장)
- FFmpeg (녹화 기능용)

단계별 설치

1. 저장소 클론

```
git clone https://github.com/your-repo/vrmeet-server.git
cd vrmeet-server
```

2. 의존성 설치

```
npm install --production
```

3. 환경 변수 설정

```
cp .env.example .env
nano .env # 프로덕션 값으로 수정
```

4. SSL 인증서 (Let's Encrypt)

```
sudo certbot certonly --standalone -d your-domain.com
```

5. PM2로 프로세스 관리

```
npm install -g pm2
pm2 start server.js --name vrmeet
pm2 save
pm2 startup
```

Nginx 리버스 프록시 설정

```
server {
    listen 443 ssl;
    server_name your-domain.com;

    ssl_certificate /etc/letsencrypt/live/your-domain.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/your-domain.com/privkey.pem;

    location / {
        proxy_pass http://localhost:8080;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_read_timeout 86400; # WebSocket 연결 유지
    }
}
```

Unity 빌드 설정

Quest (Android)

Build Settings:

- |— Platform: Android
- |— Texture Compression: ASTC
- |— Minimum API Level: 29 (Android 10)
- |— Target Architectures: ARM64

Player Settings:

- |— XR Plugin Management: OpenXR
- |— Render Mode: Multi-Pass (안정성)
- |— Graphics API: OpenGL ES3 / Vulkan

Quality Settings:

- |— Anti-Aliasing: 4x MSAA
- |— Shadow Distance: 30
- |— Texture Quality: Half Res (성능)

PCVR (Windows)

Build Settings:

- |— Platform: Windows
- |— Architecture: x86_64
- |— Compression Method: LZ4HC

Player Settings:

- |— XR Plugin Management: OpenXR
- |— Graphics API: DirectX 12
- |— Fullscreen Mode: Fullscreen Window

TURN/STUN 서버 설정

NAT 통과를 위한 필수 설정:

```
// WebRTCConfiguration.cs
public static RTCTConfiguration GetConfig()
{
    return new RTCTConfiguration
    {
        iceServers = new[]
        {
            // 무료 STUN 서버
            new RTCTIceServer { urls = new[] { "stun:stun.l.google.com:19302" } },

            // TURN 서버 (필수 - 기업 방화벽 통과)
            new RTCTIceServer
            {
                urls = new[] { "turn:your-turn-server.com:3478" },
                username = "username",
                credential = "password"
            }
        }
    };
}
```

Coturn 설치 (자체 TURN 서버)

```
# Ubuntu/Debian
sudo apt install coturn

# /etc/turnserver.conf
listening-port=3478
tls-listening-port=5349
fingerprint
lt-cred-mech
user=vrmeet:secure_password
realm=your-domain.com
cert=/etc/letsencrypt/live/your-domain.com/fullchain.pem
pkey=/etc/letsencrypt/live/your-domain.com/privkey.pem

# 시작
sudo systemctl enable coturn
sudo systemctl start coturn
```

7.2 보안 고려사항

인증 보안

요소	구현	설명
비밀번호 해시	bcrypt (12 rounds)	레인보우 테이블 공격 방지
토큰	JWT (24시간 만료)	상태 비저장 인증
Rate Limiting	5회/분 (로그인)	브루트포스 방지
HTTPS	TLS 1.3	전송 암호화

입력 검증

```
// server.js - 메시지 검증 예시
function validateMessage(message) {
  // 타입 검증
  if (typeof message.type !== 'string') return false;
  if (message.type.length > 50) return false;

  // 데이터 크기 제한
  if (message.data && message.data.length > 1024 * 1024) return false;

  // 허용된 타입만 처리
  const allowedTypes = ['room-join', 'vr-position', ...];
  if (!allowedTypes.includes(message.type)) return false;

  return true;
}
```

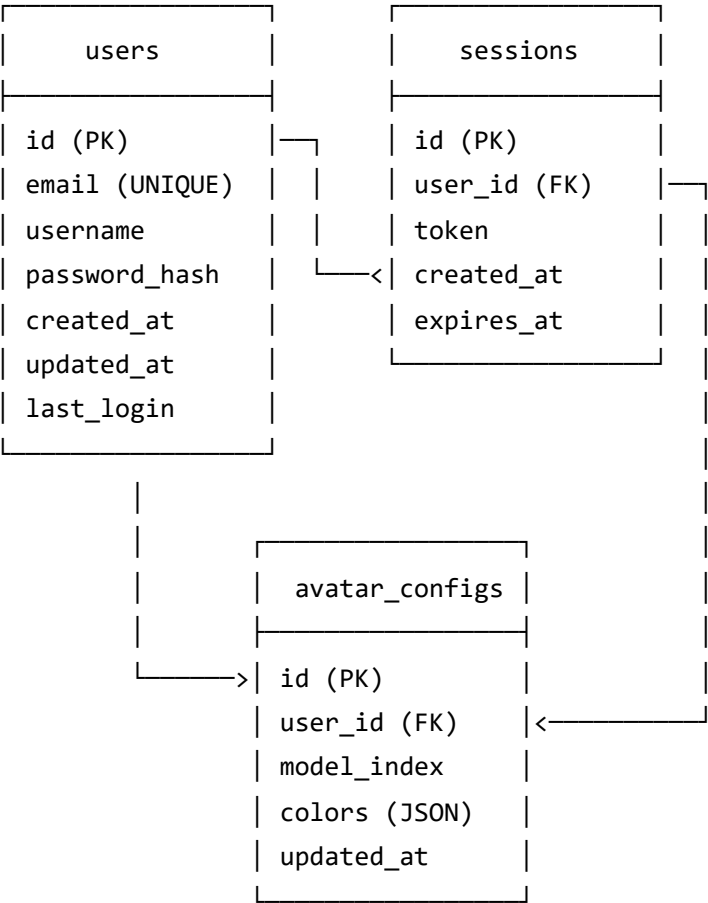
보안 체크리스트

- ✓ HTTPS/WSS 사용 (프로덕션)
- ✓ 비밀번호 bcrypt 해시
- ✓ JWT 토큰 만료 시간
- ✓ Rate limiting
- ✓ 입력 데이터 검증
- ✓ SQL Injection 방지 (준비된 문)

- ☐ E2E 암호화 (향후 계획)
- ☐ GDPR 준수 (향후 계획)

7.3 데이터베이스 스키마

ERD (Entity-Relationship Diagram)



테이블 정의

-- 사용자 테이블

```
CREATE TABLE users (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  email VARCHAR(255) UNIQUE NOT NULL,  
  username VARCHAR(50) NOT NULL,  
  password_hash VARCHAR(255) NOT NULL,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
  last_login TIMESTAMP NULL,  
  INDEX idx_email (email)  
);
```

-- 세션 테이블

```
CREATE TABLE sessions (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  user_id INT NOT NULL,  
  token VARCHAR(500) NOT NULL,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  expires_at TIMESTAMP NOT NULL,  
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,  
  INDEX idx_token (token(255)),  
  INDEX idx_user_id (user_id)  
);
```

-- 아바타 설정 테이블

```
CREATE TABLE avatar_configs (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  user_id INT UNIQUE NOT NULL,  
  model_index INT DEFAULT 0,  
  colors JSON,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE  
);
```

-- 회의 기록 테이블 (향후 확장)

```
CREATE TABLE meeting_history (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  room_id VARCHAR(10) NOT NULL,  
  host_id INT,  
  started_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  ended_at TIMESTAMP NULL,
```

```

    participant_count INT DEFAULT 0,
    recording_path VARCHAR(500) NULL,
    FOREIGN KEY (host_id) REFERENCES users(id) ON DELETE SET NULL,
    INDEX idx_room_id (room_id),
    INDEX idx_started_at (started_at)
);

```

7.4 트러블슈팅 가이드

연결 문제

문제	원인	해결 방법
WebSocket 연결 실패	방화벽/프록시	포트 8080 개방, WSS 사용
자주 연결 끊김	서버 타임아웃	Nginx proxy_read_timeout 증가
SSL 오류	인증서 문제	인증서 갱신, 체인 확인

WebRTC 문제

문제	원인	해결 방법
음성 연결 실패	NAT/방화벽	TURN 서버 설정 확인
한쪽만 들림	ICE 후보 실패	양측 네트워크 확인, TURN 강제
음질 나쁨	네트워크 불안정	대역폭 확인, 다른 코덱 시도
에코 발생	마이크 피드백	헤드폰 사용, 에코 캔슬 활성화

VR 관련 문제

문제	원인	해결 방법
낮은 프레임레이트	GPU 과부하	품질 설정 낮춤, 그림자 끄
멀미	프레임 드랍	72fps 유지, AsyncGPUReadback
컨트롤러 인식 안됨	OpenXR 설정	런타임 재시작, 드라이버 업데이트
텔레포트 안됨	레이어 설정	Teleport 레이어 bit 31 확인

문제	원인	해결 방법
그랩 안됨	인터랙션 레이어	Grab에서 Teleport 레이어 제외

화이트보드 문제

문제	원인	해결 방법
그리기 안 보임	레이어 순서	DrawingSurface가 위에 있는지 확인
동기화 안됨	boardId 불일치	같은 화이트보드 ID 사용 확인
후발 참가자 빈 화면	Request 실패	whiteboard-request 메시지 확인

로그 분석

```
// Unity 로그 위치
// Windows: %USERPROFILE%\AppData\LocalLow\CompanyName\ProductName\Player.log
// Quest: adb logcat -s Unity

// 서버 로그
pm2 logs vrmeet
tail -f /var/log/nginx/error.log
```

7.5 테스트 전략

로컬 테스트 (ParrelSync)

Unity Hub에서 ParrelSync 사용:

1. Window > ParrelSync > Clones Manager
2. "Create new clone" 클릭
3. 클론 프로젝트 열기 (자동 동기화)

테스트 시나리오:

- └─ 원본: VR 모드 (Quest Link 또는 PCVR)
- └─ 클론: 데스크톱 모드

오프라인 모드 테스트

```
// VRNetworkManager Inspector에서 :  
[Header("Debug / Offline Mode")]  
offlineMode = true           // 서버 연결 건너뛰기  
offlineAutoCreateRoom = true // 자동 룸 생성  
offlineRoomType = MeetingRoomA // 테스트할 룸 타입
```

테스트 체크리스트

연결 테스트

- ☐ 서버 연결/재연결
- ☐ 룸 생성/입장/퇴장
- ☐ 다중 클라이언트 동시 접속

VR 동기화 테스트

- ☐ 머리 위치/회전
- ☐ 양손 위치/회전
- ☐ 아바타 외형 동기화

음성 테스트

- ☐ 2인 통화
- ☐ 3인 이상 통화 (메시 토폴로지)
- ☐ Push-to-talk (V키)
- ☐ 공간 오디오 방향성

기능 테스트

- ☐ 화이트보드 그리기/지우기
- ☐ 화면 공유
- ☐ 파일 공유/프레젠테이션
- ☐ 레이저 포인터

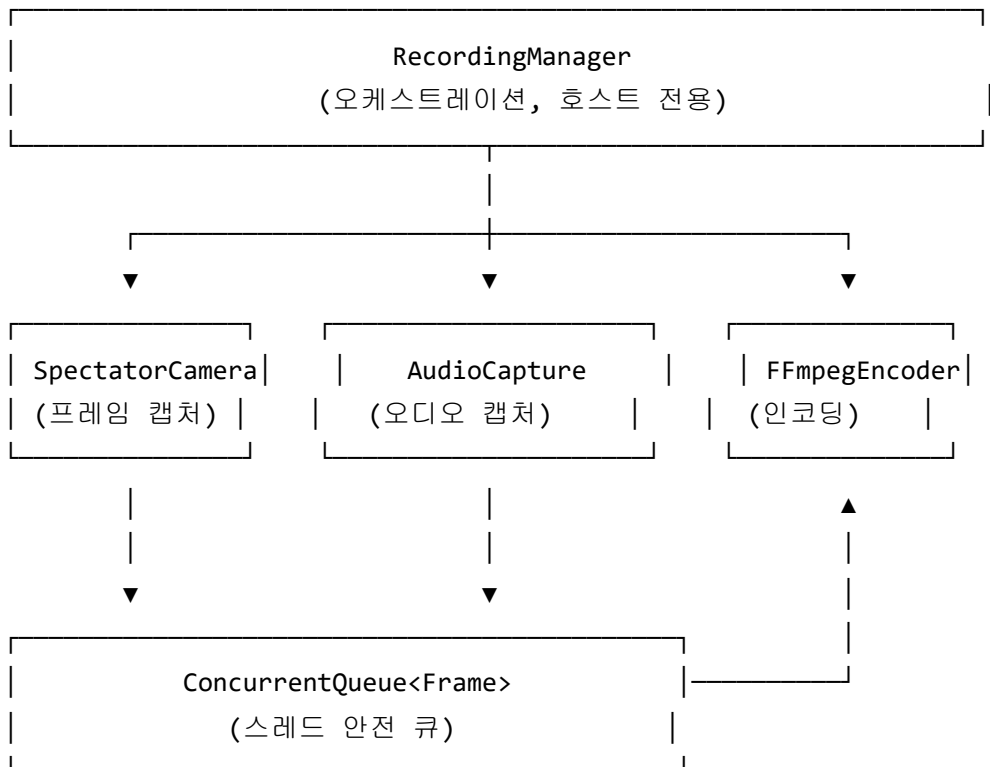
성능 테스트

```
// FPS 모니터링
void Update()
{
    float fps = 1.0f / Time.deltaTime;
    DebugManager.Log($"FPS: {fps:F1}", DebugCategory.Performance);
}

// 네트워크 지연 측정
// 서버에서 ping-pong 메시지로 RTT 계산
```

7.6 녹화 시스템 상세

아키텍처



핵심 컴포넌트

SpectatorCameraController

```
// 핵심: AsyncGPUReadback으로 VR 프레임 영향 최소화
void CaptureFrame()
{
    // GPU에서 비동기로 읽기 요청 (~0.1ms)
    AsyncGPUReadback.Request(renderTexture, 0, TextureFormat.RGB24, OnReadbackComplete);
}

void OnReadbackComplete(AsyncGPUReadbackRequest request)
{
    if (request.hasError) return;

    // 버퍼 풀에서 재사용
    byte[] buffer = _bufferPool.Rent(dataSize);
    request.GetData<byte>().CopyTo(buffer);

    // 인코딩 큐에 추가 (메인 스레드 차단 없음)
    _frameQueue.Enqueue(new FrameData(buffer, frameNumber));
}
```

3단계 파이프라인

단계	스레드	작업	부하
1. 캡처	Main	AsyncGPUReadback 요청	~0.1ms
2. 인코딩	Background	RGB → TGA 변환	~5ms
3. 쓰기	Background	File.WriteAllBytes	~2ms

설정 (RecordingSettings)

```
[Serializable]
public class RecordingSettings
{
    public int width = 1920;
    public int height = 1080;
    public int frameRate = 30;
    public int jpegQuality = 85;
    public bool captureAudio = true;
    public string outputFolder = "Recordings";
}
```

FFmpeg 인코딩

```
# 설치 (Windows)
# https://ffmpeg.org/download.html 에서 다운로드
# PATH에 추가

# 인코딩 명령 (내부적으로 실행)
ffmpeg -framerate 30 -i frame_%06d.tga -i audio.wav \
    -c:v libx264 -preset medium -crf 23 \
    -c:a aac -b:a 128k \
    -pix_fmt yuv420p \
    output.mp4
```

마커 시스템

```
public enum MarkerType
{
    Important, // 중요 순간
    Question,  // 질문
    Todo,      // 할 일
    Idea       // 아이디어
}

// 마커 추가 (모든 클라이언트에 동기화)
RecordingManager.Instance.AddMarker(MarkerType.Important, "핵심 결정 사항");
```

부록

A. 파일 구조

```
WebSocket_VR/
├── Assets/
│   ├── Scripts/           # C# 소스 코드
│   │   ├── Network/      # 네트워크 코어
│   │   ├── VR/           # VR 컨트롤러
│   │   ├── WebRTC/       # 음성 통신
│   │   ├── WhiteBoard/   # 화이트보드
│   │   ├── Sharing/      # 콘텐츠 공유
│   │   ├── UI/           # 인터페이스
│   │   └── ...
│   ├── Scenes/           # Bootstrap + Meet
│   └── Prefabs/           # Unity 프리팹
├── Server/
│   ├── server.js          # 메인 서버
│   └── src/
│       ├── database.js    # DB 연결
│       └── auth.js        # 인증
└── Packages/
    └── manifest.json      # Unity 의존성
```

B. 유용한 명령어

```
# 개발 서버 실행
cd Server && npm run dev
```

```
# 프로덕션 서버 실행
cd Server && npm start
```

```
# Quest 로그 확인
adb logcat -s Unity
```

```
# Unity 로그 확인 (Windows)
type %USERPROFILE%\AppData\LocalLow\CompanyName\ProductName\Player.log
```

C. 환경 변수 설정

서버 (.env)

```
PORT=8080
NODE_ENV=production

# 데이터베이스
DB_HOST=localhost
DB_USER=vrmeet
DB_PASSWORD=your_secure_password
DB_NAME=vrmeet_db

# 인증
JWT_SECRET=your_jwt_secret_key_here
JWT_EXPIRES_IN=24h

# CORS (선택)
ALLOWED_ORIGINS=https://your-domain.com
```

Unity 서버 URL

VRNetworkManager.cs 에서 설정:

- 개발: ws://localhost:8080
- 프로덕션: wss://your-domain.com

D. 패키지 의존성

Unity 패키지 (manifest.json)

패키지	버전	용도
com.endel.nativewebsocket	1.1.4	WebSocket 클라이언트
com.unity.webrtc	3.0.0	음성 통신
com.unity.xr.interaction.toolkit	3.2.2	VR 인터랙션
com.unity.xr.openxr	1.16.1	OpenXR 런타임

패키지	버전	용도
com.unity.xr.hands	1.7.2	핸드 트래킹
com.unity.render-pipelines.universal	17.2.0	URP 렌더링
com.veriorpies.parrelsync	1.5.2	멀티 인스턴스 테스트
com.unity.textmeshpro	3.0.9	UI 텍스트

서버 패키지 (package.json)

패키지	용도
ws	WebSocket 서버
mariadb	데이터베이스 연결
bcrypt	비밀번호 해시
jsonwebtoken	JWT 토큰
dotenv	환경 변수
uuid	고유 ID 생성

E. 단축키 요약

VR 모드

버튼	기능
A 버튼 (오른손)	레이저 포인터
그립	오브젝트 잡기
트리거	텔레포트 / UI 선택
V 키 (키보드)	Push-to-talk

데스크톱 모드

키	기능
WASD	이동
Shift	달리기
마우스 우클릭 + 드래그	카메라 회전
마우스 좌클릭	화이트보드 그리기
L	레이저 포인터
V	Push-to-talk
Esc	메뉴

F. 용어 사전

용어	설명
WebSocket	양방향 실시간 통신 프로토콜
WebRTC	브라우저/앱 간 P2P 실시간 통신
STUN	NAT 뒤의 공용 IP 발견 서버
TURN	NAT 통과 실패 시 릴레이 서버
ICE	최적의 연결 경로 찾는 프레임워크
SDP	미디어 세션 설명 프로토콜
OpenXR	VR/AR 오픈 표준 API
URP	Unity Universal Render Pipeline
AsyncGPUReadback	GPU 데이터 비동기 읽기

G. 참고 자료

공식 문서

- [Unity XR Interaction Toolkit](#)
- [Unity WebRTC](#)
- [OpenXR Specification](#)
- [WebRTC API](#)

유용한 링크

- [NativeWebSocket GitHub](#)
- [ParrelSync](#)
- [Coturn TURN Server](#)

문서 작성일: 2026년 2월 27일

Unity 버전: 6000.2.14f1

프로젝트: VRMeet - 멀티플레이어 VR 회의 솔루션